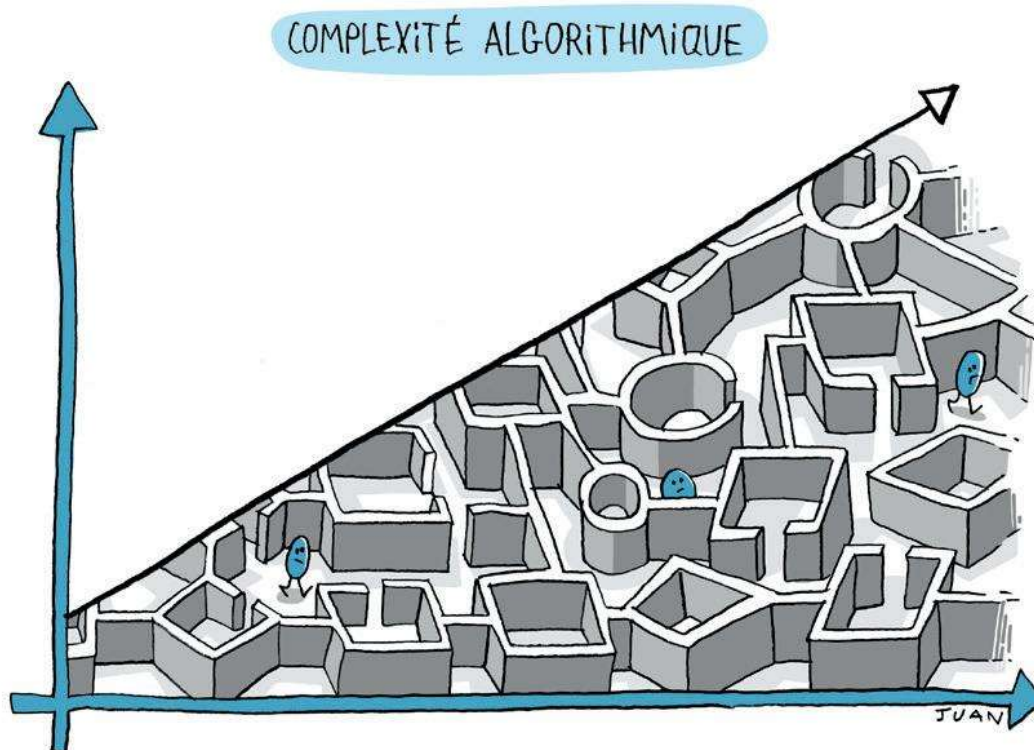


Projet de Recherche opérationnelle

Etude de la complexité

SM602



Source Image :

https://issuu.com/chumontreal/docs/lexique_intelligence_artificielle_sante_chum_eias/s/15050226

Note : après un échange avec M. JUANICO, il nous a été conseillé/recommandé de densifier les points pour des petits n et de réaliser quelques itérations de grands n .

Préambule :

Pour mieux comprendre l'importance du choix de la méthode de calcul ainsi que du langage de programmation, il nous a été demandé de réaliser une analyse de complexité des différentes méthodes vues dans ce projet, c'est-à-dire : Nord-Ouest, Balas-Hammer, marche-pied avec la proposition issue de Nord-Ouest et le marche-pied avec la proposition issue de Balas-Hammer.

Les différents temps d'exécutions calculés seront notés : θ_{NO} , θ_{BH} , t_{NO} et t_{BH} .

I- Nuages de points et courbes moyenne :

Pour comparer les différents temps d'exécution des 4 méthodes, ainsi que la somme des temps d'exécution ($\theta_{NO} + t_{NO}$ et $\theta_{BH} + t_{BH}$), on affiche le nuage de points des temps d'exécution.

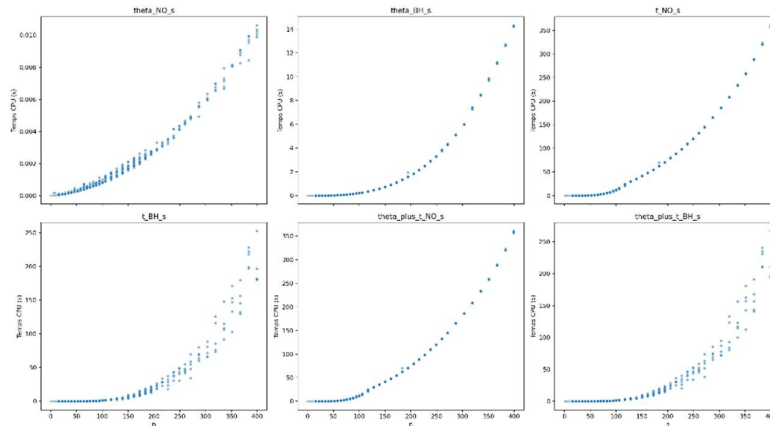


Figure 1 : Nuage de points des différentes méthodes

En analysant les valeurs du temps CPU (s) pour les différentes méthodes, on peut déjà hiérarchiser la vitesse des différents algorithmes, ce qui nous donne (du plus rapide au plus lent):

- 1) Nord-Ouest
- 2) Ballas-Hammer
- 3) Marche-pied avec Balas-Hammer
- 4) Marche-pied avec Nord-Ouest

Ce classement est totalement cohérent avec la logique des différents algorithmes.

Nord-Ouest parcourt le problème très rapidement avec très peu de calculs. Ballas-Hammer, quant à lui, optimise davantage les calculs en réduisant le nombre d'itérations nécessaires par rapport à Nord-Ouest, mais reste moins gourmand que les méthodes de marche-pied. Les méthodes du marche-pied impliquent des ajustements successifs, surtout lorsque la proposition initiale est issue de Nord-Ouest, qui ne fournit pas une base aussi équilibrée que celle de Balas-Hammer. La méthode du marche-pied avec Balas-Hammer comme point de départ limite les corrections et, par conséquent, le temps CPU, mais reste plus coûteuse que les deux premières méthodes.

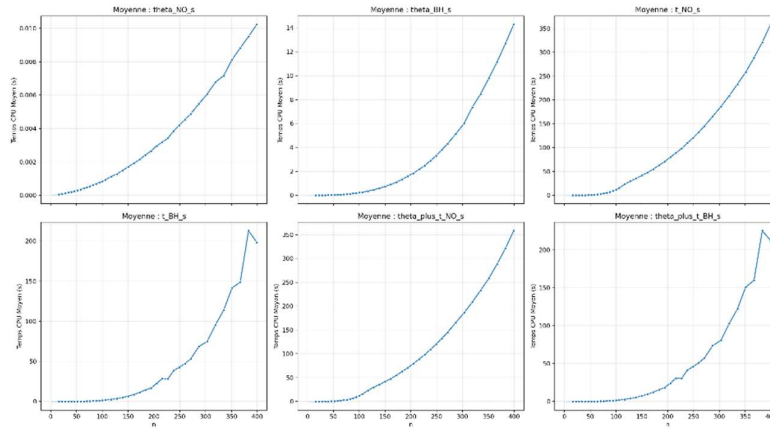
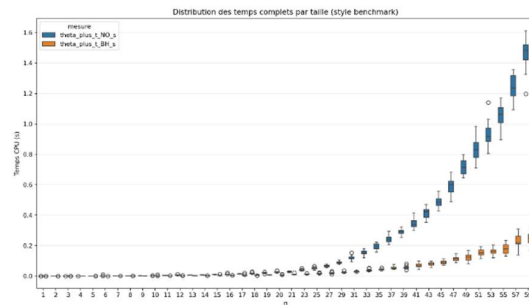


Figure 2 : Courbes des moyennes du temps d'exécution

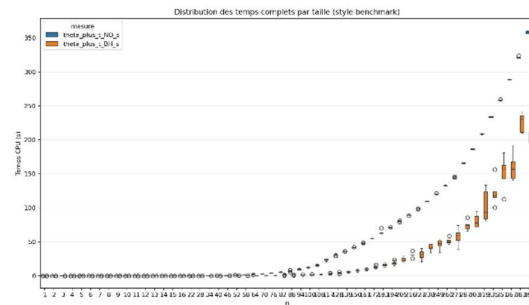
Pour avoir un meilleur aperçu des courbes, on peut réaliser le même graphe avec la moyenne des durées pour les mêmes valeurs de n .

II- Comparaison BH et NO :

Après avoir réalisé les différentes courbes, on peut aussi comparer la distribution des temps complets par taille.



$n = 60$



$n = 400$

Figure 3 & 4: Distribution des temps complets par taille

En utilisant un diagramme avec des boîtes moustaches, on remarque deux éléments distincts pour une petite valeur de n ($n \leq 60$), la méthode $\theta_{BH} + t_{BH}$ est beaucoup plus efficace que la méthode $\theta_{NO} + t_{NO}$ et qu'elle est aussi beaucoup plus stable (les boîtes moustaches montrent une bien plus grande variabilité pour la méthode avec Nord-Ouest).

Cependant, on peut aussi remarquer une autre chose : pour des problèmes de transports de petites tailles : $n \leq 20$, la différence de temps d'exécution est marginale.

Cela veut donc dire, qu'en fonction de la taille, la complexité peut être très proche. Cependant, quand n augmente (n proche de 400), la tendance change et la méthode NO devient plus stable que celle de Ballas-Hammer.

On remarque aussi que l'écart entre les deux courbes semble se réduire, même si Ballas-Hammer reste toujours plus rapide.

III- Comparaison des différentes méthodes :

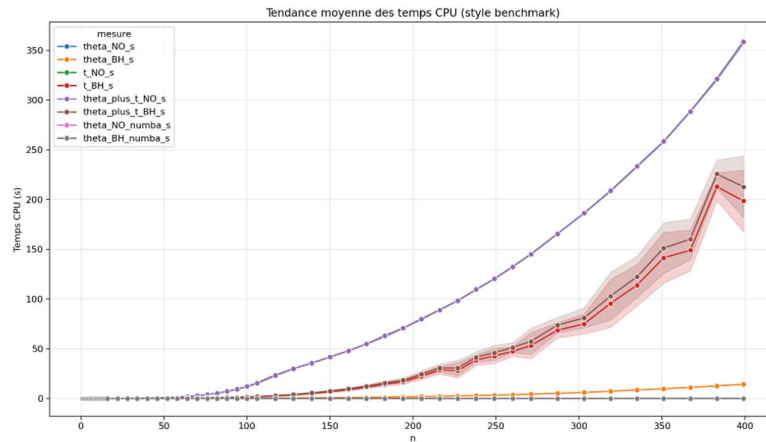


Figure 5 : Tendence moyenne des temps CPU

Après avoir comparé les deux méthodes : $\theta_{NO} + t_{NO}$ et $\theta_{BH} + t_{BH}$, on peut aussi comparer les temps moyens des différentes méthodes sur le même graphique.

On remarque que θ_{NO} reste la méthode la plus lente de toutes les méthodes, comme supposé dans la partie I. Cependant, on remarque aussi que, puisque θ_{NO} est négligeable devant t_{NO} (θ_{NO} est plusieurs milliers de fois plus rapide que t_{NO} : 0.00028 s contre 0.8 s pour $n = 50$, soit environ 2857x plus rapide).

Cela signifie, que les tracés de t_{NO} et $\theta_{NO} + t_{NO}$ sont confondus.

IV- Comparaison complexité théorique et complexité empirique :

La méthode du Nord-Ouest est la plus simple et efficace : elle parcourt la matrice une seule fois en avançant progressivement ligne par ligne et colonne par colonne. Chaque case est traitée au plus une fois, avec un nombre constant d'opérations. Comme la matrice contient n^2 cases, la complexité dans le pire des cas est en $O(n^2)$.

La méthode de Balas-Hammer est plus coûteuse car elle doit recalculer des pénalités pour toutes les lignes et colonnes encore actives avec un coût $O(n^2)$. Comme il y a environ n itérations, la complexité totale dans le pire des cas est en $O(n^3)$.

Pour la méthode de marche-pied avec les potentiels après avec Nord-Ouest, chaque itération nécessite de recalculer les potentiels et les coûts marginaux sur toute la matrice, ce qui coûte $O(n^2)$. Il faut des fois plusieurs itérations pour atteindre l'optimum (jusqu'à $O(n)$ dans le pire cas). Ce qui nous donne une complexité globale de $O(n^3)$.

Enfin, pour Balas-Hammer avec marche-pied, on multiplie le coût de l'initialisation en $O(n^2)$ avec celui de l'optimisation en $O(n^2)$. Donc, la complexité globale est $O(n^4)$ dans le pire des cas.

Tableau récapitulatif des complexités théoriques :

Méthode	Complexité dans le pire cas en notation O
Nord-Ouest	n^2
Ballas-Hammer	n^3
Nord-Ouest avec marche-pied	n^3
Ballas-Hammer avec marche-pied	n^3

On peut donc afficher la courbe théorique des complexités face aux courbes empirique pour comparer.

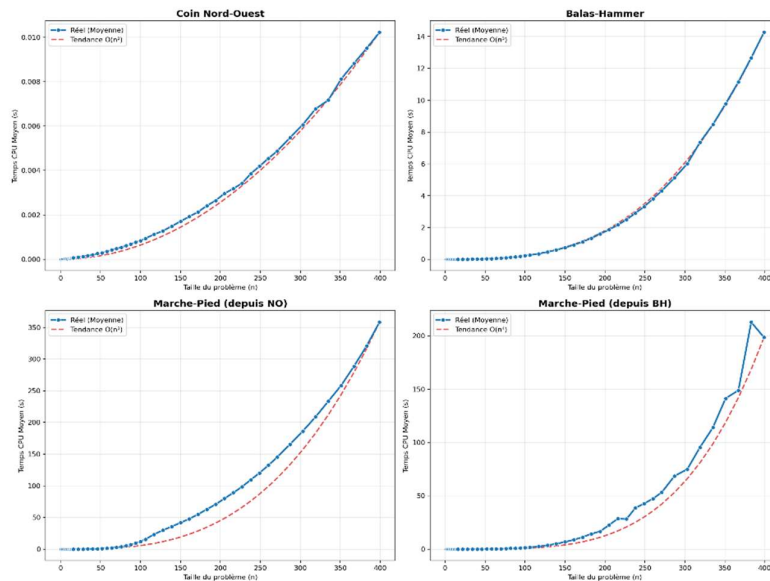


Figure 6 : Comparaison complexité théorique et empirique

On remarque que les courbes théoriques et empiriques sont très proches les unes des autres, jusqu'à être même presque confondues pour la méthode Nord-Ouest et Balas-Hammer.

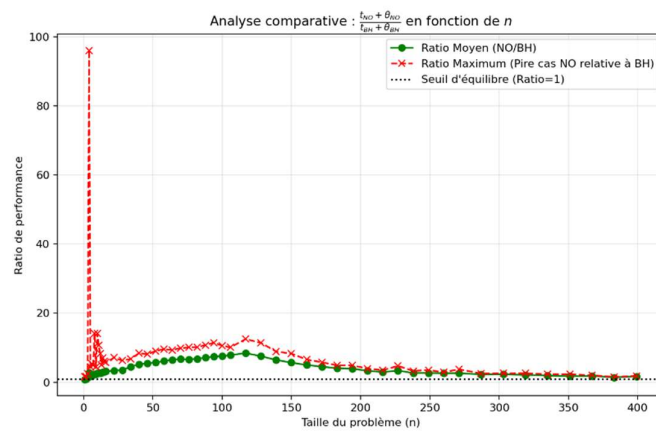


Figure 7 : Analyse comparative ratio

En analysant ce graphique, on remarque que le ratio $\frac{t_{NO} + \theta_{NO}}{t_{BH} + \theta_{BH}}$ est presque toujours supérieur à 1, ce qui montre que Nord-Ouest + marche-pied est plus coûteux que Balas-Hammer + marche-pied. L'écart est notable en moyenne et très élevé dans le pire des cas, car Nord-Ouest fournit souvent une mauvaise solution initiale, ce qui augmente le nombre d'itérations du marche-pied.

À l'inverse, Balas-Hammer démarre plus près de l'optimum. Quand n augmente, l'écart diminue, mais Balas-Hammer reste globalement plus efficace et plus robuste.

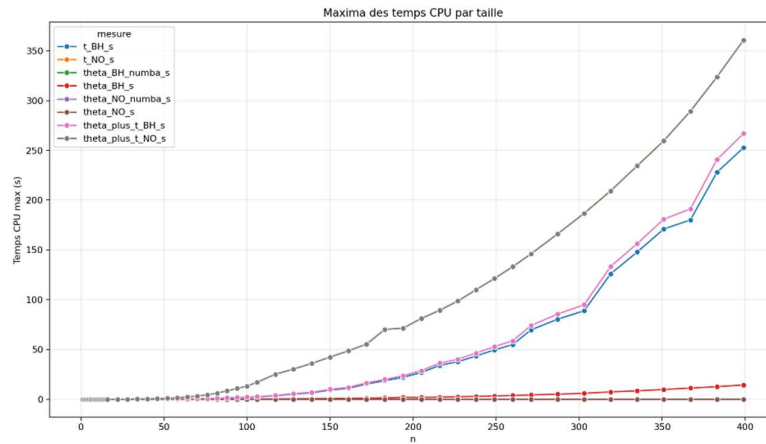


Figure 8 : Maxima des temps CPU

On retrouve bien la même tendance que ce que l'on avait analysé avant : tous les temps augmentent avec la taille de n , mais ceux liés à Nord-Ouest deviennent nettement plus élevés que ceux de Balas-Hammer. En particulier, le temps total pour Nord-Ouest croît beaucoup plus vite, ce qui confirme qu'il génère des cas très coûteux et que la méthode Nord-Ouest n'est pas forcément la meilleure pour des grandes valeurs de n .

Balas-Hammer reste plus stable : même dans le pire des cas, ses temps sont plus faibles. L'écart entre les deux méthodes s'accroît avec n , ce qui montre que Nord-Ouest est moins scalable et moins robuste, alors que Balas-Hammer maîtrise mieux la croissance du temps de calcul.

Analyse supplémentaire :

V- Comparaison mode de code :

La complexité du code est un facteur très important dans le choix du développement d'une application, d'un projet ou d'une modélisation mathématique. Nous avons déjà remarqué que le choix d'implémentation et de méthode de calcul pouvait avoir de grandes conséquences sur le temps de calcul, mais un autre facteur déterminant est le langage de programmation.

Ici, on compare du python pure avec une version Numba (un compilateur qui optimise le code python en code machine).

Après avoir recodé une nouvelle version de Ballas-Hammer et de Nord-Ouest pour que les nouvelles versions soient compatibles avec Numba, on peut relancer l'analyse de complexité sur les mêmes problèmes de transports.

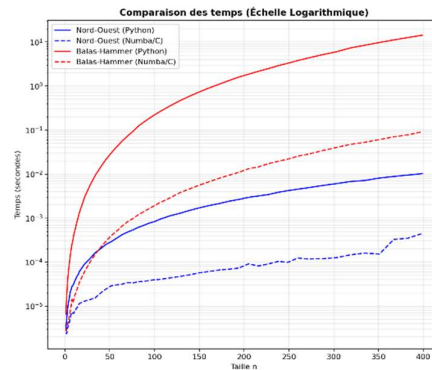


Figure 8 : Comparaison des temps python/Numba

En analysant, on se rend compte que les courbes de Ballas-Hammer et de Nord-Ouest avec Numba augmentent beaucoup plus lentement que leurs équivalents en python pure.

L'échelle logarithmique montre même que pour des valeurs de n inférieures à 35, Ballas-Hammer avec Numba est plus rapide que Nord-Ouest en python pur.

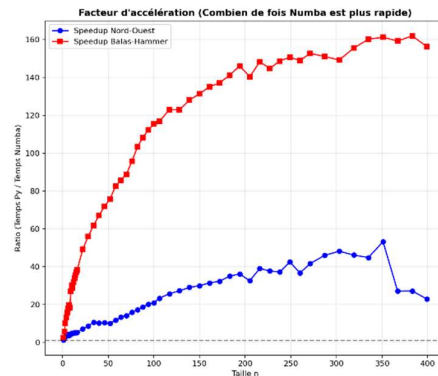


Figure 9 : Facteur d'accélération

En comparant le temps d'exécution moyens des deux modes (avec et sans Numba), on remarque que la version Numba est beaucoup plus rapide que la version normale.

En plus de cela, on remarque même que le ratio d'augmentation augmente quand n augmente, ce qui signifie que le gain apporté par Numba devient de plus en plus important lorsque la taille du problème augmente.

Cela nous montre l'importance de bien choisir le langage de programmation ainsi que la méthode de modélisation lorsque l'on veut réaliser un projet de manière efficace.